

Six Ways a Network Simulation Study Produces a Wrong Number

A self-audit of one LoRa/802.11 co-design, with the guards that caught each

Mohamed A. Farouk

AUTHBC Project — methodological companion (auto-generated from frozen results)

Abstract—Twenty years ago Kurkowski *et al.* found that none of 84 MobiHoc simulation papers referenced public code, and follow-up work reports that fewer than 15% of MANET simulation studies are reproducible. Those surveys tell us the literature has a credibility problem; they do not tell an author what, mechanically, goes wrong. We report the other half: a self-audit of a single LoRa/802.11 authentication co-design in which ten quantitative results moved or were retracted, and a taxonomy of the six distinct mechanisms responsible. Only one is fixed by running more seeds. Two were protected by a passing test suite. Three were internal contradictions between a paper and its own artifacts that every gate in the project reported green. For each class we give the concrete instance, the number it moved, and the mechanical check that now prevents it — a bootstrap over threshold crossings, an RNG-stream isolation assertion, a test that parses the paper’s $\text{\LaTeX}$ tables and compares them to the CSVs, and a test that compares the documentation’s own counts to reality. We argue that *seed discipline is necessary and badly insufficient*, and that the useful unit of reproducibility work is not the artifact but the *invariant a machine can check*.

I. INTRODUCTION

Simulation credibility in networking has been studied as a *reporting* problem. Kurkowski, Camp and Colagrosso surveyed MobiHoc 2000–2004 and found that experiments were rarely repeatable, unbiased, rigorous and statistically sound at once [1]; a decade later the VANET community reached similar conclusions [2]. The prescription that followed is familiar: publish code, report seeds, give confidence intervals.

We took that prescription seriously, and it was not enough. Over a five-week audit of one co-design study — an authenticated UAV telemetry ledger evaluated with NS-3 and two Raspberry Pi radios — ten quantitative results moved or were withdrawn. Every one had passed the project’s own gates. **Seed discipline caught four of them. It was structurally incapable of catching the other six.** Four later review passes over the write-up moved roughly thirty further numbers, and the dominant mechanism in those was not any of the five: it was corrections that had been made correctly at the source and never propagated (C6).

This paper is the taxonomy. It is deliberately a self-audit: every instance is our own, with the before and after numbers, because a defect list assembled from other people’s papers cannot say what the wrong number *was*.

II. THE FIVE CLASSES

A. C1 — A small-sample mean compared against a threshold

The known one. A capacity figure is defined as the largest N whose mean delivery still meets $V \geq 0.95$; with three seeds the mean lands on the wrong side of the threshold. **Instance:** $N_{\max}$ fell from 5 to 3 at 30 seeds; a delay crossing moved $U=2.797 \rightarrow 2.435$; a validation band endpoint moved $-1.44\% \rightarrow -0.51\%$. **Guard:** 30 seeds by default, and drivers that emit $\min/\max/\sigma$. **This is the only class more seeds fix.**

B. C2 — An unverified constant on the measurement path

Instance: a two-node broadcast experiment returned 97.45% delivery with $\sigma = 0.196$ pp over eight windows — tight, repeatable, and meaningless. 802.11 sends broadcast at the lowest basic rate, which on 2.4 GHz is 802.11b’s 1 Mb/s; a 1400 B frame then occupies 11.2 ms, so the nominal “100 fps operating point” was 118% of capacity. The measurement was saturation, not channel loss. **Why seeds cannot help:** eight windows agreed to 0.2 pp *on the wrong quantity*. Repeatability is not validity. **Guard:** state the assumption the expectation depends on, and put a discriminator in the same run — an offered-load sweep beside the point measurement located the ceiling at 501 fps and exposed it immediately.

C. C3 — A threshold applied to a mean rather than to a distribution

Instance: at the certified $N=3$, *nine of thirty* runs failed the very criterion being certified. The mean cleared 0.95; the distribution straddled it. Reported as a point, $N_{\max}=3$ has a 95% bootstrap CI of $[2,3]$; read per realisation — at least 95% of runs must individually meet V — it is **1**. **Why it is not C1:** the sample was already 30. The defect is that a reliability target was evaluated as a throughput average. **Guard:** a nonparametric bootstrap over the crossing itself, since a crossing is discrete and does not inherit a mean’s interval.

D. C4 — A configuration change that perturbs the random realisation

Instance: installing a mobility model appeared to cost 5 percentage points of delivery. NS-3 assigns RNG stream indices in object-creation order, and mobility models are constructed before senders, so the comparison arms differed in bookkeeping rather than physics. The tell was that 5 m/s and 20 m/s gave *identical* delivery — a positional effect would

scale with displacement. **Guard:** pin the senders’ streams by node id, and assert an invariant the physics forces: sent counts transmissions, so no propagation or gateway option may change it. Applied across the study’s other comparison axes, that assertion found them clean.

E. C5 — A claim wider than the experiment supports

Instance: a two-node hardware measurement with one transmitter validates airtime and link loss. It cannot validate a contention model, which describes N competing stations — and an earlier draft implied it did. Separately, a figure was quoted from the correct paper but the wrong figure, inverting a comparison. **Guard:** the discipline of writing what a result cannot show beside what it can, and **quoting the figure, not the paper.**

F. C6 — A correction that does not propagate

The class we did not anticipate, and the one with the most instances. C1–C5 describe how a number is *produced* wrongly. C6 is different: the number is corrected *once*, at its source, and the correction fails to reach every place the value already appears. The source of truth and the artifacts agree with each other; only the derived copies disagree, and they disagree silently.

Instances, all from four further review passes over a single paper:

- a frame header measured at 44 B replaced an assumed 40 B — and **five constants derived from it kept the old value.** Searching for “40” finds none of them, because none of them prints the header: they print $(M - H_f - g_a)/(b+1)$, $M/(M - H_f - g_a)$, a batch ceiling.
- a finding was **retracted** in the audit register and remained printed in a table, one hundred lines below a sentence asserting its opposite in bold;
- six three-seed artifacts were **deleted** as superseded, and a table still built on one of them matched the deleted file to three decimals;
- a threshold crossing corrected in a table caption was left uncorrected on a figure’s axis;
- a simulator upgrade documented in the paper’s own limitations was left stale in a plot title;
- a completed phase left a figure footing “pending”.

Why seeds, artifacts and code tests are all blind to it.

There is no wrong computation to find. Every artifact is correct and mutually consistent; the defect lives in the copies — prose, captions, axis labels, plot titles — which no gate reads. Two of the six had *already been found and fixed elsewhere in the same repository*, which is what makes the class distinctive: the project had the right answer written down and shipped the wrong one anyway.

Guard. Re-derive every consumer when an input changes, and treat a retraction as a `grep` obligation rather than a register entry. Mechanically: parse the prose for the constants it quotes and recompute them from the measured inputs. Note the asymmetry — a half-correct table is *harder* to catch than a

wholly wrong one, because the columns that agree with their source make the table read as verified.

III. WHAT THE GATES MISSED, AND WHY

Three of the ten corrections were *internal contradictions* in which a paper disagreed with its own artifacts while every test passed:

- a capacity table kept superseded values (233/116) after the prose was corrected to 213/100;
- a sentence stating a ratio “is stable under either threshold” sat in the same paper as a sentence warning that exactly that rounding is misleading ($3.22\times$ vs $2.42\times$);
- the abstract filed the study’s most durable result under “applications of established results” while the conclusion called it “the more durable contribution”.

None was a code defect, so no code test could see it. **The unit of reproducibility work that would have caught them is not an artifact but a machine-checkable invariant across the boundary between prose and data.** We now parse the paper’s $\text{\LaTeX}$ tables and compare every cell to the generating CSV, and parse the project documentation for asserted counts and compare them to reality. The second test found four stale claims on its first run.

Two defects were *protected by passing tests*. A unit test asserted `channel_utilisation(1) == 0.0` with the comment “a lone sender never contends” — true of contention, false of utilisation, and it had encoded the defect as intended behaviour. A green suite means the code does what the tests say, not that the tests say the right thing.

IV. PRE-REGISTRATION IS CHEAP AND IT WORKS

The study’s success criterion was committed two days before the result it judges, and both commits are public, so the ordering is checkable by a third party rather than asserted. A second pre-registration claim, on a later sub-study, had to be **withdrawn**: it cited an expectations file that had never been committed. We did not reconstruct it. Writing that file after the outcome was known would have manufactured evidence that is unfalsifiable from outside.

When a follow-up survey was later planned, its protocol — hypothesis, inclusion criteria, falsification threshold, keyword set, and the rule that unreadable sources are excluded from the denominator rather than scored as negatives — was committed in a data-free commit before the corpus existed. Within an hour it forced two corrections to our own claims: a phenomenon we had called unobserved turned out to have prior art, and a tally of “nine of nine” fell to four under the criteria we had just written down. **Both would otherwise have reached a submission.**

V. CONCLUSION

The credibility literature tells authors to publish code and report seeds. Both are necessary. In one audited study, seeds addressed four defects out of ten; the rest came from an unverified constant, a criterion applied to the wrong statistic, a

comparison confounded by RNG bookkeeping, an overreaching claim, and three contradictions between a paper and its own data. Four further passes over the write-up then found a sixth and larger class, in which the study had *already made* the correction and shipped the uncorrected copy regardless.

The transferable lesson is not a longer checklist. It is that *every claim worth making should have an invariant a machine can check*, including the claims a paper makes about itself — and that the most valuable checks are the ones that can return an answer you did not want. Ours did, ten times.

REFERENCES

- [1] S. Kurkowski, T. Camp, and M. Colagrosso, “MANET simulation studies: the incredibles,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 9, no. 4, pp. 50–61, 2005.
- [2] “VANETs’ research over the past decade: overview, credibility, and trends,” *ACM SIGCOMM Computer Communication Review*, 2018.